

Design: MCP-Orchestrated ERP Intelligence Layer for Harmonise Inventory and Booking Workflows

1. Purpose and scope

This design defines the architecture of an MCP-based orchestration layer that sits between Claude and the currently supplied Harmonise-backed stock/common APIs for inventory retrieval, catalogue navigation, and product detail enrichment. Within the supplied contract, the public REST surface consists of `/api/v1/common/departments`, `/api/v1/stock/categories`, `/api/v1/stock/product-catalogue`, and `/api/v1/stock/products`. Booking retrieval remains part of the target architecture, but because no booking Swagger or sample booking payload has been supplied here, booking-specific contracts in this report should be treated as proposed internal/future interfaces, not as documented facts of the current public API surface. The purpose of the layer is not to replace Harmonise, nor to duplicate Claude's conversational ability, but to create a controlled, auditable, tool-driven retrieval and reasoning boundary that translates natural-language requests into precise API calls, normalized business evidence, and grounded responses.

MCP is designed for exactly this class of integration: servers expose tools with schemas so language models can interact with external systems safely and predictably, and the MCP spec explicitly centers tool invocation and JSON Schema validation as core protocol concepts.

([Model Context Protocol](#))

2. Business problem being solved

The business issue is not simply “connect Claude to an API.” The actual problem is that user requests are conversational, ambiguous, and often incomplete, while ERP systems are deterministic, parameter-sensitive, and structured around identifiers, line items, and backend schemas. The architecture therefore must solve five operational gaps simultaneously:

1. **Product-variant resolution:** users may refer to a catalogue product by partial product name, but execution frequently needs to resolve down to a specific variant SKU, because the Harmonise stock model is product-centric at the top level and variant-centric at the actionable level.
2. **Hierarchical expansion:** useful answers often require moving across hierarchy levels — for example, from department/category filters, to product catalogue entries, to variants, and then to detailed variant fields such as rates, dimensional flags, stock by state, and component allocations.

3. **Noise suppression:** raw stock payloads contain structural metadata such as `departmentId`, `subDepartmentId`, `categoryId`, `variations`, `variants`, `optionIds`, nested `details`, and paging fields; these are useful for orchestration but too verbose for direct model consumption without normalization.
4. **Ambiguity handling:** multiple partial matches must be surfaced safely rather than guessed, especially where one product has multiple variants or where variant names differ from parent product names.
- Grounded completion: the final answer must remain faithful to the retrieved JSON, with traceability, confidence, and clear distinction between directly observed fields and derived interpretations.

This means the MCP layer is best understood as an **AI-facing evidence broker**. It converts conversational intent into validated retrieval actions, shapes the output into business-relevant evidence blocks, and hands only the needed context back to Claude.

3. System context and responsibility boundaries

The architecture involves six logical actors:

- **User:** asks general questions in natural language and may be vague or uncertain.
- **Claude:** interprets user intent, selects the correct MCP tools, manages conversational interaction, and composes the final natural-language answer.
- **MCP server:** exposes the controlled tool surface, validates parameters, resolves entities, orchestrates retrieval, normalizes payloads, manages short-term task state, and validates groundedness.
- **Harmonise API:**
 - i) authoritative source for departments, stock categories, product catalogue entries, product variants/SKUs, and detailed variant metadata.
 - ii) authoritative source for booking, quote, and booking-line, inclusive of products (i) → This is not implemented as now.
- **Azure services:** host runtime, cache state, secure secrets, and collect telemetry.

The critical design principle is that **Harmonise and Inventory remain systems of record**, while the MCP server is a transient decision and shaping layer. It may cache and summarize, but it must never become an alternate source of truth.

4. Core architectural model

At a high level, the solution is a layered architecture:



MCP Architecture

The reason this structure works is that Claude should not be asked to infer low-level API mechanics. Instead, Claude interacts with a compact tool surface, while the orchestration layer manages the retrieval mechanics and evidence shaping. This is aligned with MCP's tools model, where a server exposes named tools with schemas that models can invoke to access external systems. ([Model Context Protocol](#))

5. Azure runtime architecture

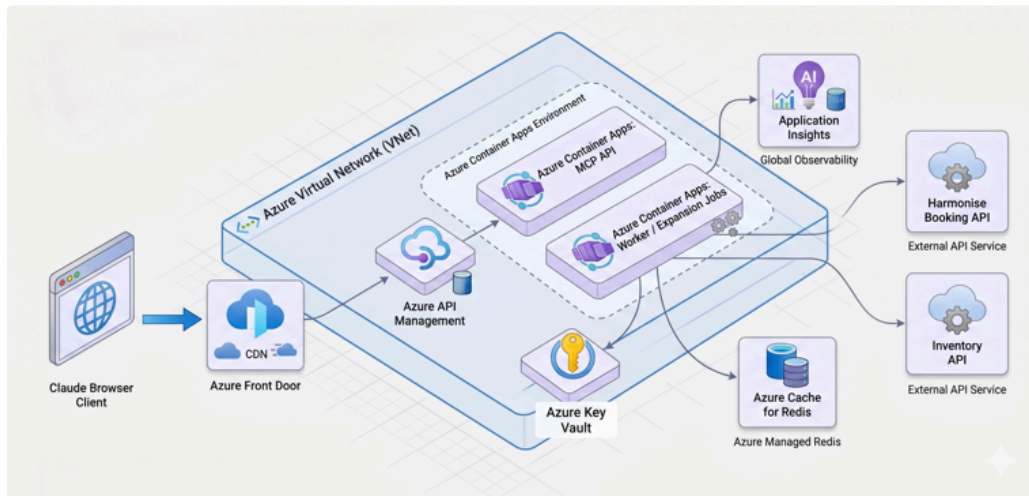
The Azure hosting layer should be treated as the operational wrapper around the MCP service. The main runtime components are:

- **Azure Container Apps** for the MCP API runtime and optional worker containers.
- **Azure API Management** as the API gateway and policy layer.
- **Azure Front Door** as the public entry point with edge routing and protection.
- **Azure Managed Redis** for short-lived working memory and response caching.
- **Azure Key Vault** for secrets, certificates, and connection strings.
- **Azure Monitor / Application Insights** for traces, request telemetry, failures, and diagnostics.

Azure Container Apps is specifically suited to this kind of service because it runs containerized applications in a managed environment, handles infrastructure/runtime concerns for container apps, and organizes workloads inside a Container Apps environment that acts as a secure boundary for apps and jobs. API Management is Azure's gateway and lifecycle platform for

exposing, securing, and governing APIs. Key Vault is Azure's secure store for secrets, keys, and certificates. Azure Managed Redis provides a managed in-memory Redis-based store for low-latency state and cache workloads. Application Insights is Azure Monitor's application performance monitoring capability and supports OpenTelemetry-based observability. Azure Front Door provides the global edge entry layer for HTTP/HTTPS traffic. ([Microsoft Learn](#))

This produces the following deployment view:



Azure Deployment

6. Logical decomposition inside the MCP server

The MCP server should be decomposed into six internal modules.

6.1 Request interpreter

This module receives the MCP tool call, validates the envelope, and turns the invocation into an internal execution context. It does not perform business retrieval itself.

6.2 Resolver and matcher

This module determines which business object the user likely means. It resolves:

- booking vs product intent
- identifier vs descriptive reference
- exact vs fuzzy match strategies
- clarification requirements when confidence is low

6.3 Retrieval adapters

These are deterministic connectors to:

- `/api/v1/common/departments` for department and sub-department lookup
- `/api/v1/stock/categories` for stock category retrieval

- `/api/v1/stock/product-catalogue` for paged catalogue search using `search`, `departmentId`, and `categoryId`
- `/api/v1/stock/products` for exact or near-exact product retrieval by id or sku, including variant detail payloads when returned
- future/internal extensions such as booking, pricing, logistics, richer media services, or availability services beyond the currently supplied stock contract

Adapters know endpoint structure, retry policy, timeout rules, parameter mapping, pagination handling, and response-shape transformations.

6.4 Normalizer

This module converts raw backend JSON into a clean internal shape:

- canonical field names
- unit normalization
- type cleaning
- removal of irrelevant fields
- preservation of provenance metadata

6.5 Task-state memory

This holds short-lived cross-step execution state, such as:

- selected booking candidate
- already-resolved line items
- pending clarification options
- current working assumptions
- expansion progress

6.6 Validation engine

This verifies that:

- the retrieval matched the intended entity
 - the final response only uses supported fields
 - ambiguity is either resolved or explicitly surfaced
 - the output confidence meets policy
-

7. Tool architecture

The exposed MCP tool surface should be small, business-centered, and stable. It should also mirror the actual supplied stock/common contract closely, so that the model works with evidence that exists in the backend rather than with invented abstractions.

7.1 Stock and catalogue tool family

These tools operate on the currently documented Harmonise stock/common surface.

`stock.get_departments`

Wraps `/api/v1/common/departments` and returns departments plus optional sub-departments for controlled filtering and metadata enrichment.

`stock.get_categories`

Wraps `/api/v1/stock/categories` and returns paged category metadata for browsing and filtering.

`stock.search_catalogue`

Wraps `/api/v1/stock/product-catalogue` and supports paged search using the currently documented backend parameters: `page`, `pageSize`, `search`, `departmentId`, and `categoryId`.

`stock.get_product`

Wraps `/api/v1/stock/products` and retrieves product records by exact `id` or `sku`, preserving product-level and variant-level structure.

`stock.get_variant_evidence`

An MCP-level composition tool that resolves a chosen product/variant into a normalized evidence block derived from the `/api/v1/stock/products` response, especially the nested `variants[].details` structure.

`stock.compare_variants`

An MCP-level comparison tool, not a direct backend endpoint, used to compare multiple resolved variants side by side after retrieval.

7.2 Booking tool family

These tools operate on quote and booking intelligence.

`booking.search` (*proposed/internal*)

Search bookings by quote ID, date range, customer hint, location, or status.

`booking.get_booking` (*proposed/internal*)

Retrieve a single booking header and line-item structure.

`booking.expand_items` (*proposed/internal*)

Resolve booking line items against stock tools and return enriched item evidence.

7.3 Shared utility tools

These tools support orchestration rather than direct business retrieval.

`resolver.disambiguate_candidates`

Return a ranked list of product or variant candidates and a structured clarification question.

`session.get_state`

Expose current task-state to continue a multi-step workflow.

`session.clear_state`

Reset operational memory for the active task.

The key design rule is that tools should return **task-usable evidence**, not raw ERP dumps, while still preserving enough structure to distinguish product-level records from variant-level facts.

8. Parameter architecture

Parameter quality determines retrieval quality. The same user utterance can map to several possible backend calls, so the parameter layer needs strong controls.

8.1 Parameter principles

- Prefer exact backend-supported identifiers when present, especially product `id`, variant `sku`, `departmentId`, and `categoryId`.
- Preserve the distinction between **backend-supported query parameters** and **MCP-side derived matching logic**; do not pretend the backend supports free-form attribute filters when it does not.
- Forbid silent coercion of incompatible inputs.
- Normalize strings before downstream matching.
- Handle paging explicitly because catalogue and product responses are paged and return `page`, `pageSize`, `totalCount`, and `totalPages`.
- Attach metadata such as `request_id`, `session_id`, and `trace_id`.
- Return `confidence`, `matched_on`, and `normalization_notes` with every non-trivial result.

8.2 Inventory search schema example

```
1 {
2   "name": "stock.search_catalogue",
3   "description": "Search the Harmonise stock catalogue using the
4   currently documented backend parameters.",
5   "input_schema": {
6     "type": "object",
7     "properties": {
8       "page": {
9         "type": "integer",
10        "minimum": 1,
11        "default": 1
12      },
13      "pageSize": {
14        "type": "integer",
15        "minimum": 1,
16        "maximum": 100,
17        "default": 20
18      },
19      "search": {
20        "type": "string",
21        "description": "Free-text catalogue search string supported by
22        /api/v1/stock/product-catalogue."
23      },
24      "departmentId": {
25        "type": "integer"
26      },
27      "categoryId": {
28        "type": "string",
29        "format": "uuid"
30      }
31    },
32    "additionalProperties": false
33  }
34 }
```

8.3 Booking search schema example

```
1 {
2   "name": "booking.search",
3   "description": "Search bookings by quote id, date, customer hint,
4   location, or status.",
5   "input_schema": {
6     "type": "object",
7     "properties": {
8       "quote_id": { "type": "string" },
9       "date_from": { "type": "string", "format": "date" },
10      "date_to": { "type": "string", "format": "date" },
11      "customer_hint": { "type": "string" },
12      "location": { "type": "string" },
13      "status": { "type": "string" },
14      "limit": {
15        "type": "integer",
16        "minimum": 1,
17        "maximum": 10
18      }
19    },
20    "additionalProperties": false
21  }
22 }
```


9. Retrieval and matching model

Retrieval should use a staged model that maximizes determinism first and uses approximate logic only when necessary.

9.1 Stage 1: deterministic resolution

Apply in this order:

- exact product `id` when known
- exact variant `sku` when known
- exact `departmentId` / `categoryId` constrained retrieval when the user has already narrowed scope
- exact normalized product name against retrieved catalogue entries
- exact variant name match within a retrieved product's `variants[]`

9.2 Stage 2: approximate resolution

If deterministic resolution fails, compute a hybrid confidence score using:

- lexical overlap over product names and variant names
- fuzzy string similarity on `name` and `sku`
- variant-option evidence derived from `variations`, `options`, and `optionIds` where available
- department/category narrowing from `departmentId`, `subDepartmentId`, and `categoryId`
- field-specific importance weights that prioritize exact SKU and exact product/variant name evidence over looser semantic hints

A typical weighted score looks like:

```
score = lexical + fuzzy + attribute overlap + exact field bonuses +  
recency/status modifiers
```

This is especially important for cases like:

- “white Tiffany chairs”
- “that round table from the Richmond booking”
- “the fridge item with low temp”
- “the quote for the event next Friday”

9.3 Clarification threshold

The architecture should not continue silently when candidate separation is weak. If:

- the top confidence is below policy threshold, or
- the gap between the top two candidates is too small,

the resolver should return a clarification payload instead of a best-guess result.

9.4 Clarification response example

```
1  {
2    "status": "needs_clarification",
3    "question": "I found multiple likely matches for 'white chair'.
4    Which one did you mean?",
5    "options": [
6      {
7        "candidate_id": "itm_001",
8        "label": "Tiffany Chair - White",
9        "matched_on": ["name", "colour"],
10       "confidence": 0.83
11     },
12     {
13       "candidate_id": "itm_014",
14       "label": "Folding Chair - White",
15       "matched_on": ["colour", "category"],
16       "confidence": 0.79
17     }
18   ]
19 }
```

This mechanism is crucial because it moves ambiguity out of the model's hidden reasoning and into an explicit, inspectable interface.

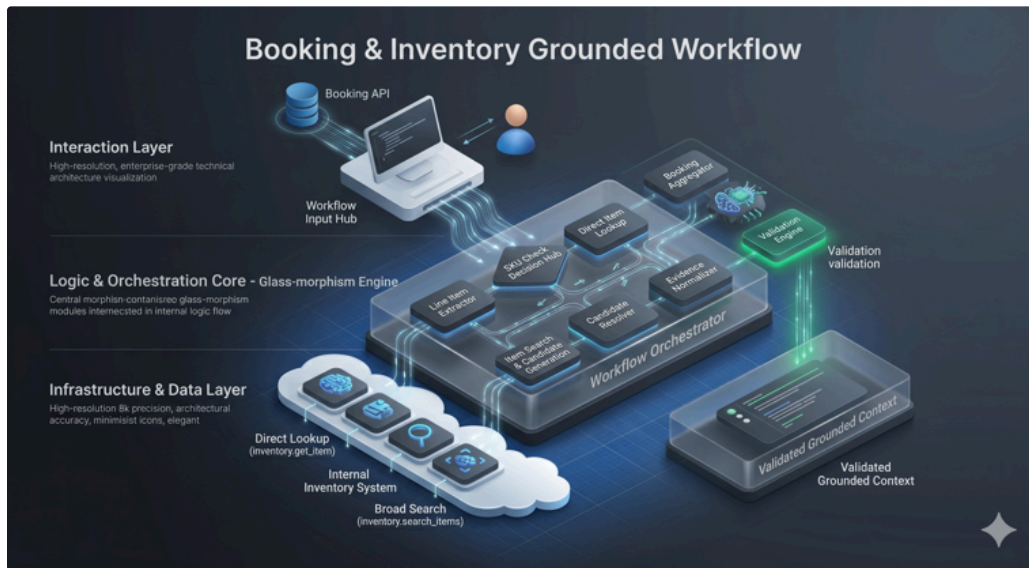
10. Booking expansion model

A booking rarely answers the whole user question by itself. Most useful business responses require expanding line items into enriched inventory evidence.

10.1 Expansion steps

1. Retrieve booking header and line items.
2. Extract each line's SKU or item hint.
3. Resolve each line against inventory.
4. Enrich each line with key item fields only.
5. Return a combined payload with resolved and unresolved sections.

10.2 Expanded flow



Evidence Orchestration

10.3 Design implications

- Expansion should be parallelized per line item when latency matters.
- Inventory enrichment should be limited to fields that are actually present in the supplied stock contract, such as product and variant identifiers, SKU, variant name, department/category identifiers, rates, dimensional flags, `length`, `width`, `height`, `stock` counts by state, `totalStock`, `lastUpdatedDate`, `imageFileName`, `cost`, and component allocations.
- Fields such as `colour`, `temperature`, media URL, or availability status should not be treated as standard evidence unless another source contract explicitly supplies them.
- Unresolved or low-confidence items must be retained in a separate list rather than dropped.

10.4 Expanded booking example

```
1 {
2   "booking_id": "bk_10492",
3   "quote_id": "Q-10492",
4   "resolved_items": [
5     {
6       "line_id": "1",
7       "requested_sku": "f1-la-la-lam-1-ble",
8       "product_id": "d2a50000-0e48-c047-a948-08dde35c3ea0",
9       "product_name": "Laminate Timber Floor",
10      "variant_id": "d2a50000-0e48-c047-a956-08dde35c3ea0",
11      "variant_name": "Bleached Oak",
12      "sku": "f1-la-la-lam-1-ble",
13      "quantity": 120,
14      "departmentId": 2,
15      "subDepartmentId": 14,
16      "categoryId": "d2a50000-0e48-c047-f36d-08dde35c3ea4",
17      "details": {
18        "generalRate": 70,
```

```

19     "expoRate": 70,
20     "dimensional": true,
21     "canBeSoldInPortions": false,
22     "length": 1,
23     "width": 1,
24     "height": 0,
25     "vicStock": 3200,
26     "vicHirable": 0,
27     "nswStock": 250,
28     "nswHirable": 0,
29     "qldStock": 0,
30     "qldHirable": 0,
31     "totalStock": 3450,
32     "lastUpdatedDate": "2025-09-09",
33     "imageFileName": null,
34     "cost": 18,
35     "components": [
36         {
37             "componentId": "d2a50000-0e48-c047-a956-08dde35c3ea0",
38             "quantity": 1
39         }
40     ],
41     },
42     "confidence": 0.98
43 }
44 ],
45 "unresolved_items": [],
46 "validation": {
47     "all_requested_skus_resolved": true,
48     "response_confidence": 0.96
49 }
50 }

```

11. Normalization layer

The normalization layer exists because backend payloads are often unfit for direct model use. It should impose a canonical internal format across all sources.

11.1 Responsibilities

- canonical field naming
- data-type normalization
- measurement/unit normalization
- empty/null handling
- attribute flattening
- dictionary-based alias registration
- provenance retention
- field suppression for non-essential noise

11.2 Example raw-to-normalized transformation

Raw backend payload may contain:

- inconsistent casing
- duplicate image arrays
- mixed units
- internal foreign-key references
- ERP-specific status codes
- null-heavy nested objects

Normalized output should contain:

- `product_id`
- `product_name`
- `variant_id`
- `variant_name`
- `sku`
- `departmentId`
- `subDepartmentId`
- `categoryId`
- `optionIds`
- `details`
- `stock_by_location`
- `pricing`
- `dimensions`
- `component_allocations`
- `paging_context` (when relevant)
- `provenance`

11.3 Example normalized inventory item

```
1 {
2   "product_id": "d2a50000-0e48-c047-a948-08dde35c3ea0",
3   "product_name": "Laminate Timber Floor",
4   "variant_id": "d2a50000-0e48-c047-a956-08dde35c3ea0",
5   "variant_name": "Bleached Oak",
6   "sku": "fl-la-la-lam-1-ble",
7   "departmentId": 2,
8   "subDepartmentId": 14,
9   "categoryId": "d2a50000-0e48-c047-f36d-08dde35c3ea4",
10  "optionIds": [
11    "d2a50000-0e48-c047-a8b8-08dde35c3ea0"
12  ],
13  "pricing": {
14    "generalRate": 70,
15    "expoRate": 70,
16    "cost": 18
```

```

17 },
18 "dimensions": {
19   "dimensional": true,
20   "canBeSoldInPortions": false,
21   "length": 1,
22   "width": 1,
23   "height": 0
24 },
25 "stock_by_location": {
26   "vicStock": 3200,
27   "vicHirable": 0,
28   "nswStock": 250,
29   "nswHirable": 0,
30   "qldStock": 0,
31   "qldHirable": 0,
32   "totalStock": 3450
33 },
34 "lifecycle": {
35   "isActive": true,
36   "startDate": "2025-08-25",
37   "endDate": null,
38   "lastUpdatedDate": "2025-09-09"
39 },
40 "component_allocations": [
41   {
42     "componentId": "d2a50000-0e48-c047-a956-08dde35c3ea0",
43     "quantity": 1
44   }
45 ],
46 "media": {
47   "imageFileName": null
48 },
49 "normalization": {
50   "canonical_entity_level": "variant",
51   "resolved_from": ["product_name", "variant_name", "sku"]
52 },
53 "provenance": {
54   "source": "/api/v1/stock/products",
55   "retrieved_at": "2026-04-21T15:42:10Z"
56 }
57 }

```

12. Prompt and harness design

Prompting should be treated as system behavior specification, not creative writing. The model should operate under a strict action policy:

- use tools before guessing
- prefer exact identifiers first
- ask for clarification when ambiguity persists
- do not invent SKU, quantity, availability, or booking status
- answer only from retrieved evidence
- explicitly separate known facts from unknowns

Anthropic's prompt guidance emphasizes clear instructions, examples, structured formatting, and prompt chaining for complex workflows; these are directly applicable to this ERP

orchestration scenario. ([Model Context Protocol](#))

12.1 Harness structure

The Claude-side system prompt should define:

- role
- tool-use policy
- ambiguity policy
- answer style
- stopping conditions
- definition of done

12.2 Example hidden control scaffold

```
1 <role>ERP booking and inventory assistant</role>
2 <goal>Resolve user requests using MCP tools and answer only from
  validated evidence.</goal>
3 <tool_policy>
4   Prefer exact identifiers when available.
5   Use search tools only when exact lookup is unavailable.
6   Ask clarifying questions if multiple candidates remain plausible.
7 </tool_policy>
8 <answer_policy>
9   Be concise, direct, and grounded.
10  State missing or uncertain fields explicitly.
11 </answer_policy>
```

12.3 Few-shot examples

The harness should include a small number of high-value examples:

- partial product name leading to clarification
- ambiguous booking lookup by date/location
- booking expansion to retrieve item details
- safe refusal to guess when no candidate meets threshold

13. Memory architecture

The report context already assumes conversational history is handled in Claude's browser experience. The MCP layer therefore should implement **operational memory**, not duplicate chat memory.

13.1 Memory tiers

Request memory

Exists only for the current tool execution:

- input normalization
- candidate lists
- per-step plan state

Task-state memory

Short-lived memory across multiple tool calls in the same user task:

- selected booking
- pending clarification options
- resolved line items
- expansion progress
- current objective

This is the right fit for Redis because it is low-latency, ephemeral, and optimized for state/cache workloads. Azure Managed Redis is a managed Redis service intended for exactly this style of in-memory state and cache use. ([Microsoft Learn](#))

Durable audit memory

Longer-lived records for:

- traces
- failed retrieval attempts
- accepted user clarifications
- alias improvements
- evaluation samples
- audit evidence

This store is not used as the business source of truth. It exists for operational insight, replay, testing, and governance.

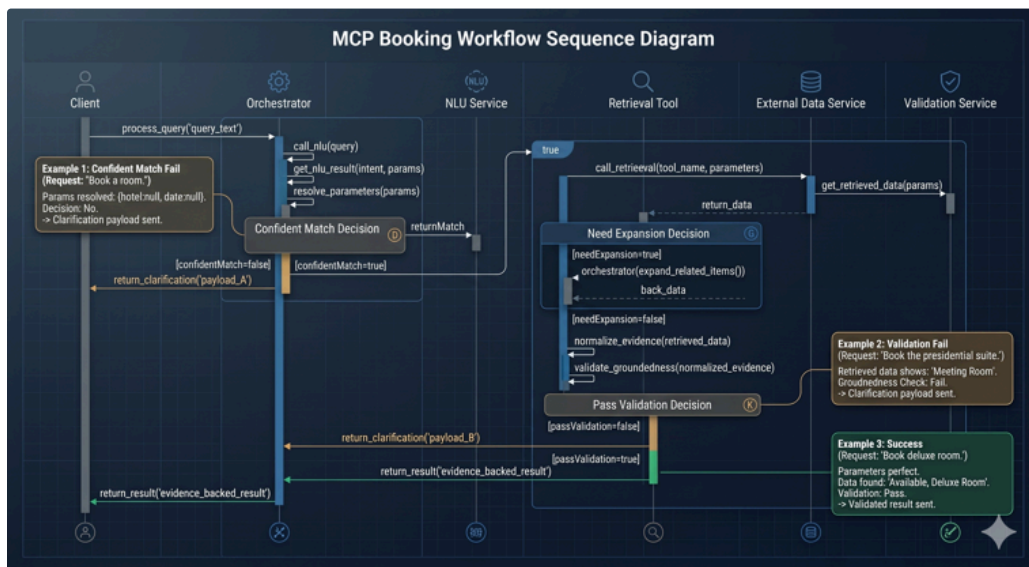
13.2 Memory policy

- keep only what the current task requires
 - use TTLs aggressively
 - never let task-state override source-of-truth data
 - persist only trace-worthy operational events
 - separate sensitive data handling from general telemetry
-

14. Planning model

Planning should be explicit and lightweight. The goal is not to build a second autonomous planner inside the MCP server, but to enforce a stable execution path.

14.1 Planning state machine



Schedule Diagram

14.2 Definition of done

A task should be treated as complete only when:

- the intended entity is resolved
- all required related evidence is retrieved
- normalized evidence supports the answer
- unresolved ambiguity is explicitly surfaced
- confidence passes policy threshold
- trace data is recorded

Without this discipline, the system risks fluency without correctness.

15. Validation architecture

Validation is the mechanism that stops a plausible-sounding answer from outrunning the evidence.

15.1 Validation layers

Schema validation

Checks that incoming and outgoing tool payloads conform to their schemas.

Entity validation

Confirms that:

- the selected booking is the one actually retrieved
- resolved SKUs belong to that booking
- claimed item attributes exist in inventory evidence

Grounding validation

Compares the answer draft to evidence using:

- token overlap
- field coverage checks
- fuzzy phrase alignment
- unsupported-claim detection

Confidence policy

The final output should include a confidence calculation derived from:

- resolver confidence
- field coverage completeness
- entity consistency
- unsupported-claim penalties

15.2 Validated response envelope example

```
1 {
2   "answer": "The resolved stock item is Laminate Timber Floor /
   Bleached Oak (SKU fl-la-la-lam-1-ble). The retrieved detail shows
   generalRate 70, expoRate 70, totalStock 3450, dimensional=true, and
   stock distributed across VIC, NSW, and QLD. No media URL was returned
   in the supplied detail payload.",
3   "confidence": 0.93,
4   "grounding": {
5     "product_fields": ["id", "name", "departmentId",
6       "subDepartmentId", "categoryId"],
7     "variant_fields": ["id", "name", "sku", "optionIds"],
8     "detail_fields": [
9       "generalRate",
10      "expoRate",
11      "dimensional",
12      "canBeSoldInPortions",
13      "length",
14      "width",
15      "height",
16      "vicStock",
17      "nswStock",
18      "qldStock",
19      "totalStock",
20      "lastUpdatedDate",
21      "imageFileName",
22      "cost",
23      "components"
```

```
23     ]
24   },
25   "missing_or_uncertain": [
26     "No booking evidence was used in this response",
27     "No direct media URL was returned",
28     "No colour field exists in the supplied detail contract"
29   ]
30 }
```

This approach ensures the answer payload itself remains inspectable.

16. Security boundary and controls

Because MCP tools create a bridge between a model and enterprise systems, the security model must be strict. The MCP specification itself explicitly calls out security and trust considerations because the protocol enables arbitrary data access and code execution paths if left uncontrolled. ([Model Context Protocol](#))

16.1 Boundary controls

- expose only approved tools
- keep backend endpoints private where possible
- enforce authentication and authorization at gateway and service layers
- validate every tool payload against schema
- limit prompt-accessible operations to read-oriented, scoped capabilities
- redact secrets and sensitive backend fields before model exposure

16.2 Secret management

API keys, credentials, certificates, and connection strings should be stored in Key Vault rather than application configuration files. Azure Key Vault is explicitly designed for secure storage and controlled access to secrets, keys, and certificates. ([Microsoft Learn](#))

16.3 Public ingress and gateway policy

Front Door and API Management should separate public exposure from backend service execution:

- Front Door handles external edge ingress
 - API Management handles gateway policies, access control, throttling, and backend routing
 - Container Apps hosts the execution service
 - Harmonise APIs remain behind the MCP boundary
-

17. Observability and auditability

This architecture needs observability at both the application and agentic levels:

- inbound request trace
- tool-call trace
- retrieval timing
- ambiguity rate
- cache hit rate
- validation failure rate
- unsupported-claim incidents
- backend error rates

Azure Monitor / Application Insights supports application telemetry and OpenTelemetry-based instrumentation, making it suitable for tracing the full request-to-tool-to-backend path.

([Microsoft Learn](#))

17.1 Audit event structure

```
1 {
2   "request_id": "req_8f8c3",
3   "session_id": "sess_a12e",
4   "user_intent": "Find the booking for the Richmond event and list
5   what chairs are included",
6   "plan": [
7     "search booking",
8     "resolve booking candidate",
9     "expand line items",
10    "filter chairs",
11    "return concise answer"
12  ],
13  "tool_calls": [
14    {
15      "tool": "booking.search",
16      "args": {
17        "location": "Richmond",
18        "limit": 5
19      },
20      "result_summary": "2 candidates found"
21    },
22    {
23      "tool": "booking.get_booking",
24      "args": {
25        "quote_id": "Q-10492"
26      },
27      "result_summary": "booking resolved"
28    },
29    {
30      "tool": "booking.expand_items",
31      "args": {
32        "booking_id": "bk_10492"
33      },
34      "result_summary": "2/2 items resolved"
35    }
36  ],
37  "final_confidence": 0.96,
```

```
37   "completed": true
38 }
```

17.2 Operational metrics

The most important metrics are:

- percentage of exact-resolution requests
- clarification rate
- average booking expansion latency
- mean tool-call count per completed task
- validation pass rate
- groundedness failure rate
- unresolved item rate per booking
- user correction rate after first answer

These metrics reveal not only system health but retrieval quality.

18. Example business flows

Flow A: partial product lookup

User asks:

“Do we have the white Tiffany chairs?”

Execution path:

1. Claude selects `inventory.search_items`
2. MCP normalizes query
3. resolver runs hybrid search
4. one strong candidate found
5. normalized item evidence returned
6. Claude answers with availability and attributes only if present in evidence

Flow B: ambiguous booking lookup

User asks:

“Show me the Richmond quote from last Friday.”

Execution path:

1. Claude selects `booking.search`

2. multiple results returned
3. MCP returns clarification bundle
4. Claude asks user which candidate they meant
5. user confirms
6. Claude calls `booking.get_booking`

Flow C: booking-to-item expansion

User asks:

“What items are in quote Q-10492 and what colour are the chairs?”

Execution path:

1. Claude calls `booking.get_booking`
 2. booking line items retrieved
 3. Claude calls `booking.expand_items`
 4. MCP enriches each SKU via inventory
 5. validation confirms evidence coverage
 6. Claude answers from enriched item set
-

19. Core design principles governing the solution

This architecture is governed by the following principles:

Source-of-truth discipline

Harmonise and inventory services remain authoritative. The MCP layer does not author business facts.

Small tool surface

Fewer, clearer tools outperform many thin wrappers because they reduce model confusion and improve testability.

Deterministic first, fuzzy second

Exact retrieval must always be attempted before approximate resolution.

Clarify instead of guessing

Ambiguity is surfaced as a structured interface, not hidden inside model reasoning.

Normalize before reasoning

Raw ERP payloads should not be fed directly to the model when a normalized evidence layer can reduce noise.

Validate before answering

Every answer should be tied to retrieved evidence and checked against schema and entity consistency.

Observe every step

Tool calls, resolution paths, and failures should be traceable for support and evaluation.

20. Architecture summary

The resulting architecture is a disciplined MCP orchestration layer that converts conversational ERP requests into validated business evidence. Claude remains the conversational and reasoning interface. The currently evidenced authoritative systems in this document are Harmonise's public stock/common APIs for departments, categories, catalogue search, and product detail retrieval. The MCP server provides the missing middle: structured tools, parameter control, product/variant disambiguation, evidence normalization, short-lived task memory, validation, and auditability. Booking orchestration remains part of the target design, but in this report it should be explicitly presented as dependent on a separate booking contract that has not yet been supplied.